

اليوم 63 - Ansible و Puppet و Chef و Terraform

في عالم أتمتة الشبكات الحديثة، هناك العديد من الأدوات المتاحة لإدارة التهيئة (Configuration Management) ونشر البنية التحتية كبرنامج (Infrastructure as Code). كل أداة من هذه الأدوات لها فلسفتها الخاصة وطريقتها في العمل ونقاط قوتها وضعفها. في هذا الدرس الأخير من دورة CCNA، سنستعرض أربعاً من أهم هذه الأدوات: Ansible و Puppet و Chef و Terraform. فهم هذه الأدوات سيمكنك من اختيار الأداة المناسبة لاحتياجات مؤسستك، وسيمنحك نظرة شاملة على مشهد أتمتة الشبكات. هذه المعرفة أساسية لأي مهندس شبكات يريد الانتقال من الإدارة اليدوية التقليدية إلى الإدارة الآلية الحديثة.

Ansible هي أداة إدارة تهيئة من Red Hat، وتعتبر الأكثر شعبية في أتمتة الشبكات. الميزة الأكبر لـ Ansible هي أنها Agentless، أي لا تحتاج إلى تثبيت أي برنامج إضافي (Agent) على الأجهزة التي تديرها. هذا أمر مهم جداً في بيئات الشبكات حيث لا يمكنك غالباً تثبيت برامج إضافية على المحولات والموجهات. Ansible تتصل بالأجهزة مباشرة عبر SSH (للأجهزة الشبكية) أو WinRM (لأجهزة ويندوز) وتنفذ الأوامر عن بعد. هذا النموذج يسمى Push Model، حيث يقوم خادم Ansible (Control Node) بدفع التهيئة إلى الأجهزة المستهدفة. Ansible تستخدم YAML لكتابة ملفات التشغيل (Playbooks) وهي سهلة القراءة والفهم حتى لغير المبرمجين. Playbook يحتوي على قائمة من المهام (Tasks) التي تنفذها وحدة (Module) معينة. لسيسكو، توفر Ansible وحدات خاصة مثل ios_config و ios_command لتنفيذ أوامر التهيئة وعرض الأوامر على أجهزة iOS.

Puppet هي أداة إدارة تهيئة أقدم من Ansible وتستخدم نموذج Pull Model. في Puppet، يتم تثبيت Agent على كل جهاز مدار. هذا الـ Agent يتصل بشكل دوري (عادة كل 30 دقيقة) بخادم Puppet Master لسحب (Pull) أحدث تهيئة له وتطبيقها. إذا كان هناك أي اختلاف بين الحالة الحالية للجهاز والتهيئة المطلوبة (Desired State)، يقوم Agent بتصحيح الفرق تلقائياً. هذا النموذج مناسب للبيئات التي تحتوي على آلاف الأجهزة ولا تريد أن يتحمل خادم واحد عناء الاتصال بجميعها. Puppet تستخدم لغة خاصة بها تسمى Puppet DSL (Domain-Specific Language) تشبه إلى حد كبير لغة Ruby، وتكتب التهيئات في ملفات بامتداد pp. Manifests. مفهوم Desired State في Puppet يعني أنك تصف كيف يجب أن يكون الجهاز، والنظام يتأكد من تحقق هذا الوصف باستمرار. هذا مفهوم قوي لكنه يتطلب تعلم لغة جديدة. Puppet تدعم أجهزة الشبكات من سيسكو عبر modules خاصة مثل cisco_ios و cisco_aci.

Chef هي أداة إدارة تهيئة أخرى تستخدم نموذج Pull Model مشابه لـ Puppet. تتكون Chef من ثلاثة مكونات رئيسية: Chef Workstation حيث يقوم المطور بكتابة التهيئات، و Chef Server الذي يخزن التهيئات (Cookbooks) وبيانات الأجهزة، و Chef Client (Agent) المثبت على كل جهاز مدار. يتم كتابة التهيئات في Chef باستخدام Ruby DSL وتسمى Cookbooks و Recipes. Recipes تحتوي على قائمة من الموارد (Resources) التي تصف الحالة المطلوبة للجهاز، مثل "الخدمة X يجب أن تكون عاملة" أو "الملف Y يجب أن يكون موجوداً بمحتوى معين". Chef تعتبر أكثر تعقيداً من Ansible و Puppet لأنها تتطلب فهم لغة Ruby وبنية Chef Server/Client. لكنها تمنح مرونة كبيرة للمطورين الذين يعرفون Ruby. في عالم الشبكات، Chef أقل انتشاراً من Ansible و Puppet، لكنها مدعومة على أجهزة سيسكو باستخدام cookbooks خاصة.

Puppet: Pull model, Agent-based, الأسهل للبدء. Ansible: Push model, Agentless, YAML Playbooks الأكثر تعقيداً والأكثر مرونة للمطورين.

Terraform من HashiCorp هي أداة مختلفة عن الأدوات السابقة. بينما Ansible و Puppet و Chef هي أدوات إدارة تهيئة Terraform (Configuration Management) هي أداة (Infrastructure as Code) تركز على نشر وإدارة البنية التحتية بأكملها (وليس فقط تهيئة الأجهزة الموجودة). الفرق الأساسي: أدوات CM تفترض أن الجهاز موجود بالفعل وتقوم بتهيئته، بينما Terraform يمكنها إنشاء الجهاز نفسه (مثلاً، استدعاء API لإنشاء جهاز افتراضي في AWS أو Azure) ثم تهيئته. Terraform تستخدم لغة HCL (HashiCorp Configuration Language) وهي لغة تصريحية (Declarative) تصف الحالة النهائية المطلوبة للبنية التحتية. أنت تقول "أريد 3 خوادم افتراضية في AWS من نوع t2.medium مع هذا الـ Security Group"، و Terraform يقوم بإنشائها وتعديلها وحذفها حسب الحاجة. Terraform تستخدم Providers لكل منصة: AWS Provider، Azure Provider، Cisco DNA Center Provider، Cisco ACI Provider، F5 BIG-IP Provider وغيرها الكثير. أمر terraform plan يظهر التغييرات التي ستنفذ قبل تطبيقها، وأمر terraform apply ينفذها.

لنأخذ مثالاً عملياً على Ansible مع سيسكو. playbook بسيط لتهيئة واجهة على جهاز سيسكو يمكن أن يبدو كالتالي. أولاً، نكتب ملف Inventory يحتوي على عناوين IP للأجهزة وبيانات تسجيل الدخول. ثم نكتب Playbook بصيغة YAML يحتوي على مهمة واحدة تستخدم وحدة ios_config. يتم تطبيق الأمر ip address على الواجهة المحددة. الـ Playbook يمكن تشغيله على 10 أجهزة أو 100 جهاز بنفس السهولة، فقط بتعديل ملف Inventory. أنشيل ستتصل بكل جهاز عبر SSH، وتنفذ الأوامر، وتجمع النتائج. يمكن استخدام خيار --check لتجربة التشغيل دون تطبيق فعلي (Dry Run)، وخيار --diff لرؤية الفرق بين التهيئة الحالية والجديدة.

مثال على Ansible Playbook لتهيئة جهاز سيسكو:

```
---
- name: Configure Cisco Router
  hosts: routers
  gather_facts: no
  connection: network_cli

  tasks:
    - name: Configure interface GigabitEthernet0/1
      ios_config:
        lines:
          - ip address 10.0.0.1 255.255.255.0
          - no shutdown
        parents: "interface GigabitEthernet0/1"
```

مثال على Terraform لإنشاء مورد في Cisco DNA Center:

مثال على Terraform مع Cisco DNA Center:

```
terraform {
  required_providers {
    dnacenter = {
      source = "cisco-open/dnacenter"
    }
  }
}

provider "dnacenter" {
  username = var.dnac_username
  password = var.dnac_password
  base_url = var.dnac_url
}

resource "dnacenter_device" "example" {
  ip_address    = ["192.168.1.1"]
  snmp_version  = "v2c"
  snmp_community = "public"
}
```

CI/CD (Continuous Integration / Continuous Deployment) هو مفهوم أساسي في الـ DevOps يمكن تطبيقه على الشبكات. في CI/CD للشبكات (NetDevOps)، يتم تخزين تكوينات الشبكة في (Git) (GitHub/GitLab). أي تغيير يتم عبر Pull Request (طلب دمج) يمر بمراحل اختبار تلقائية: فحص بناء الجملة (Syntax Check)، اختبار على شبكة اختبار (Staging)، تحليل الأمان، ومراجعة الأقران. بعد الموافقة والاختبار الناجح، يتم تطبيق التغيير تلقائياً على شبكة الإنتاج (Production) في نافذة زمنية محددة. هذا يضمن أن كل تغيير يتم اختباره بدقة قبل الوصول إلى الإنتاج، مما يقلل بشكل كبير من مخاطر انقطاع الخدمة. أدوات CI/CD مثل Jenkins و GitLab CI و GitHub Actions يمكنها دمج Ansible و Terraform في pipeline واحد متكامل. مثلاً: Push إلى Jenkins -> Git يلتقط التغيير -> Terraform plan -> إذا كان التغيير آمناً، يشغل Ansible playbook على شبكة الاختبار -> بعد النجاح، يشغل Ansible على الإنتاج.

اختيار الأداة المناسبة لأتمتة الشبكات يعتمد على عدة عوامل. Ansible هي الخيار الأفضل لمعظم مهندسي الشبكات المبتدئين في الأتمتة: سهولة التعلم، لا تحتاج إلى وكيل، لغة YAML البسيطة، ومكتبة واسعة من الوحدات لأجهزة سيسكو. Puppet مناسبة للمؤسسات الكبيرة التي تفضل نموذج Desired State وتريد ضمان امتثال الأجهزة للسياسات بشكل مستمر. Chef مناسبة للفرق التي تمتلك خبرة قوية في Ruby وتبني أنظمة معقدة. Terraform هي المكمل المثالي لـ Ansible: استخدم Terraform لإنشاء البنية التحتية (الخواادم الافتراضية، الشبكات الافتراضية، أجهزة الشبكة السحابية) واستخدم Ansible لتهيئة هذه الأجهزة بعد إنشائها. هذا المزيج يغطي دورة حياة البنية التحتية بالكامل من الإنشاء إلى التهيئة إلى الصيانة.

في الختام، هذا الدرس يختتم رحلة CCNA الخاصة بنا. لقد بدأنا من أساسيات الشبكات وبروتوكولات التوجيه والتبديل، مروراً بالأمان والشبكات اللاسلكية، وانتهاءً بأتمتة الشبكات والأدوات الحديثة. كل ما تعلمته هو أساس متين يبنى عليه. لكن تذكر أن CCNA هي

مجرد البداية، عالم الشبكات واسع ومتجدد باستمرار. الشبكات المعرفة بالبرمجيات (SDN)، وأتمتة الشبكات، والذكاء الاصطناعي في الشبكات، والحوسبة السحابية، والأمن السيبراني، كلها مجالات تتطور بسرعة وتحتاج إلى تعلم مستمر. ننصحك بمتابعة شهادات أكثر تخصصاً مثل CCNP Enterprise أو CCNP Security أو DevNet Associate حسب اهتمامك ومسارك المهني. الأهم من الشهادة هو الفهم العميق والقدرة على تطبيق ما تعلمته في حل مشاكل حقيقية. استمر في التعلم والتجربة، وشكراً لك على متابعة هذه الدورة.